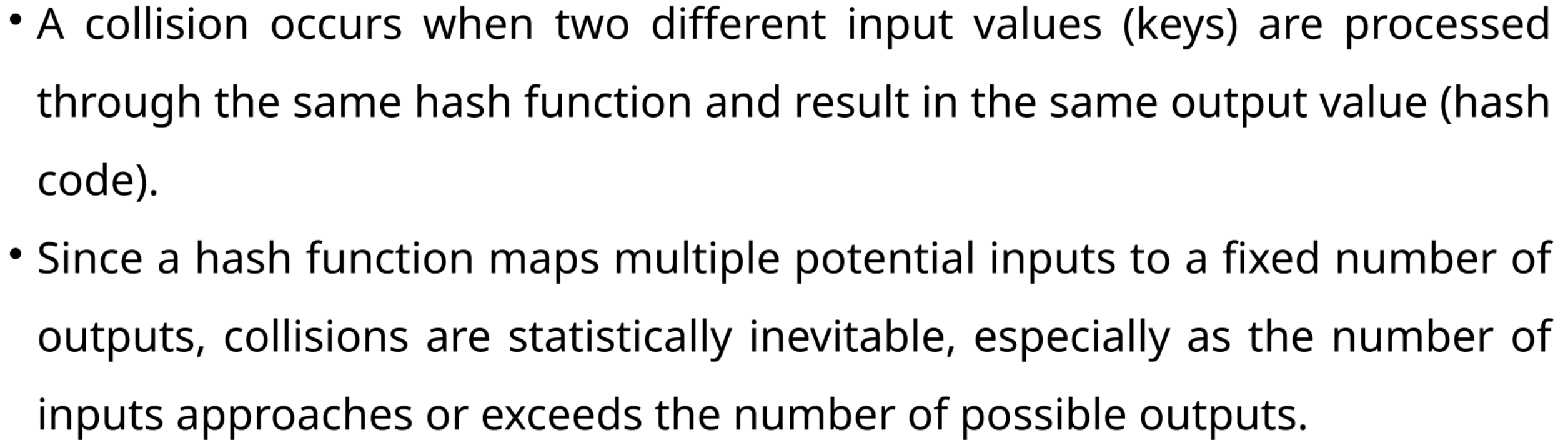
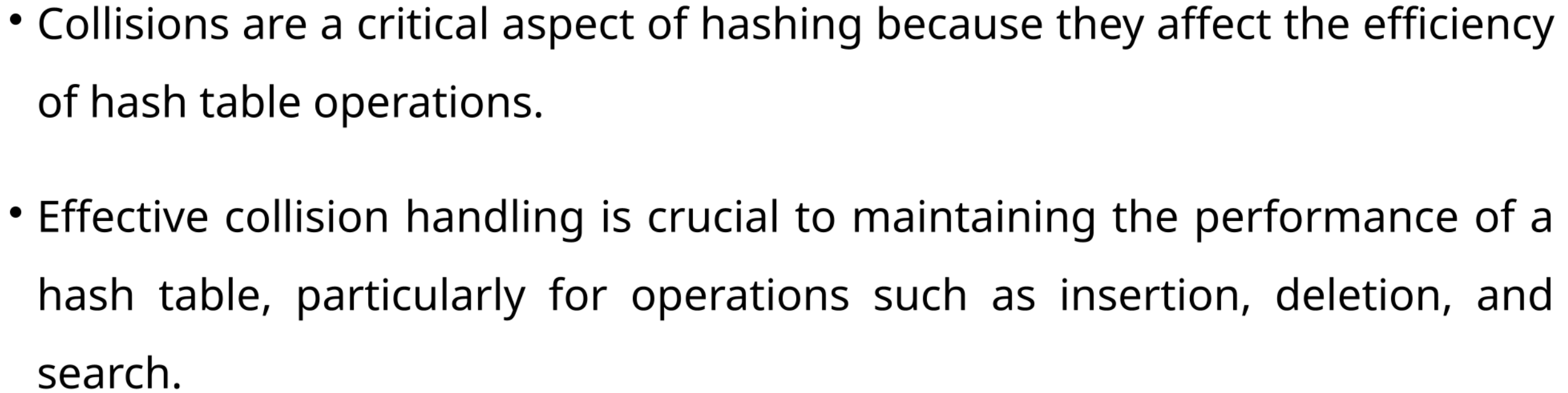


Data Structures

Dilip Maripuri
Associate Professor
Department of Computer Applications









- **Open Addressing (Closed Hashing)**
 - In open addressing, all elements are stored within the hash table itself.
 - When a collision occurs, the hash table searches for the next available slot using a probing sequence.
 - The main types of open addressing include:
 - Linear Probing
 - Quadratic Probing
 - Double Hashing



Data Structures

Hashing – Collision Resolution Techniques



- **Linear Probing**

- **Initial Insertion**

- When inserting a new key, the hash table calculates the hash index using a hash function.
 - If the slot at the calculated hash index is unoccupied, the key is placed there.



Data Structures

Hashing – Collision Resolution Techniques



- **Linear Probing**
 - **Handling Collisions**
 - If the initial hash index is already occupied by another key (a collision occurs), linear probing searches for the next available slot by moving sequentially through the table, one slot at a time.
 - This process continues until an empty slot is found.



Data Structures

Hashing – Collision Resolution Techniques



- **Linear Probing**
 - **Delete Operation**
 - Deleting a key is more complex because simply removing a key can break a chain of probes.
 - To handle deletions correctly, the key is removed, and then keys in the subsequent slots are reinserted using the original hash function to ensure all keys remain accessible.

- **In hash tables using linear probing**
 - **Search for an existing key:** You start at the calculated hash index and move sequentially through the table until you find the key or an empty slot, which indicates the key is not present.
 - **Search for a non-existing key:** You start at the hash index and follow the same probing sequence used during insertion until you find an empty slot or wrap around to your starting point, indicating the key is not in the hash table.



Data Structures

Hashing – Collision Resolution Techniques



- **Hash Function:** $\text{hash}(\text{key}) = \text{key} \bmod 12$
- **Final Hash Table State after Insertions:**

[_, 1, 13, 3, 4, 16, 15, 7, 19, 9, 21, _]



Current State of the Hash Table

[_, 1, 13, 3, 4, 16, 15, 7, 19, 9, 21, _]

- **Delete Operation**

- Search for Key 19
 - Compute Hash Index: ($\text{hash}(19) = 19 \bmod 12 = 7$)
- Search Process:
 - Start at index 7.
 - Check position 7, find 7.
 - Continue linear probing since 7 is not 19.
 - Move to index 8, where 19 is found.



- **Delete Operation**

Current State of the Hash Table

[_, 1, 13, 3, 4, 16, 15, 7, 19, 9, 21, _]

- Deletion:

- Once the key (19) is found at index 8, it can be marked as deleted.
- In hash tables using linear probing, simply emptying the slot could break the probe chain for other keys.
- A common practice is to mark the slot as "deleted" rather than completely clearing it.
- This way, the probe chain for other elements like 9 and 21, which might depend on the continued probing past the deleted element, is not broken.
- [_, 1, 13, 3, 4, 16, 15, 7, DEL, 9, 21, _]



- **Post-Deletion Search:**

- If we were to search for keys like 9 or 21, the search algorithm would continue past the DEL marker as if the key were still there, ensuring that any elements placed during the original insertion due to probing are still discoverable.

- **Re-insertion:**

- Should another key need insertion and hash to an index leading up to or beyond 8, the DEL marked slot can be reused to insert a new key, effectively recycling the deleted space.

- Deleting keys in a hash table that uses linear probing requires careful management to ensure that the search algorithm remains effective:
 - **Preserving the Chain:**
 - Marking slots as "deleted" rather than making them completely empty helps maintain the integrity of the probe sequence.
 - **Cleaning Up:**
 - Occasionally, it might be useful to rehash or clean up the table, especially if there are many deleted slots, to improve efficiency and avoid unnecessary probing.



- Let's define the hash function using the position of each character in the alphabet (where 'a' = 1, 'b' = 2, ..., 'z' = 26).

- We will sum these values for all characters in a string and then take the modulus by 12:

- $\text{hash}(\text{key}) = \text{sum}(\text{alphabet_positions}(\text{Character})) \bmod 12$



Data Structures

Hashing – Collision Resolution Techniques



- To ensure a diverse distribution, we will use: "apple", "banana", "cherry", "date", "elder", "fig", "grape", "honey", "kiwi", "lime"
- Hash Table Initialization : Start with an empty hash table:
 - [_ , _ , _ , _ , _ , _ , _ , _ , _ , _ , _ , _]
- **Insertions : Insert "apple"**
 - **Compute Hash Index :**
 - "apple" $\rightarrow (1+16+16+12+5 = 50)$
 - Hash index: $(50 \bmod 12 = 2)$
 - **Insertion :** Position 2 is empty.
 - Table after insertion: [_ , _ , "apple", _ , _ , _ , _ , _ , _ , _ , _ , _]



Initialize a hash table:

- Let TableSize be the size of the hash table.
- Initialize the table with all slots set to NULL.

Define the hash function:

- ```
- Function Hash(key):
 return key % TableSize
```



# Data Structures

## Linear Probing Algorithm



Define the insert function:

```
- Function Insert(key):
 index = Hash(key)
 start_index = index
 while table[index] is not NULL and table[index] is not DELETED:
 index = (index + 1) % TableSize
 if index == start_index:
 print "Hash table is full"
 return
 table[index] = key
 print "Key inserted at index", index
```





# Data Structures

## Linear Probing Algorithm



Define the search function:

```
- Function Search(key):
 index = Hash(key)
 start_index = index
 while table[index] is not NULL:
 if table[index] == key:
 print "Key found at index", index
 return index
 index = (index + 1) % TableSize
 if index == start_index:
 print "Key not found"
 return -1
 print "Key not found"
 return -1
```

### Define the delete function:

- Function Delete(key):

```
index = Search(key)
```

```
if index != -1:
```

```
table[index] = DELETED
```

```
print "Key deleted from index", index
```

```
else:
```

```
print "Key not found, cannot delete"
```



# THANK YOU

---

**Dilip Maripuri**  
**Associate Professor**  
**Department of Computer Applications**  
**dilip.maripuri@pes.edu**